

Übungsblatt 2

Architektur von Web-Anwendungen

5430UE Web and Data Engineering

Prof. Dr. Michael Granitzer

24 März 2026

Lernziele

- **Ressourcen** (CPU, RAM, Storage, Netzwerk) und ihre Engpässe verstehen
- **Skalierung** unterscheiden: vertikal vs. horizontal, Vor- und Nachteile
- **Architekturstile** (monolithisch, n-Tier, Microservices, ereignisgesteuert) und ihre Trade-offs einordnen
- **Verteilte Systeme** und deren Herausforderungen (CAP, Konsistenz, Latenz) erfassen
- Designprinzipien **SoC**, **SRP**, **DRY** auf Web-Architekturen anwenden

Modul A

Ressourcen und Skalierung

Übung 2.A.1: Ressourcen-Engpässe identifizieren

Ein Online-Shop beobachtet folgende Symptome:

Symptom	Vermuteter Engpass (CPU / RAM / Storage / Netzwerk)
Antwortzeiten steigen bei komplexen Preisberechnungen	?
Server stürzt ab wenn 50.000 Sessions gleichzeitig aktiv sind	?
Produktbilder laden sehr langsam	?
API-Requests von mobilen Clients laufen regelmäßig in Timeouts	?

Begründen Sie jede Zuordnung in einem Satz.

Lösung

Symptom	Engpass
Langsame Preisberechnungen	CPU — rechenintensive Operationen erschöpfen die Verarbeitungskapazität
Absturz bei 50.000 Sessions	RAM — Session-Daten im Arbeitsspeicher; bei ~1 MB/Session = 50 GB RAM-Bedarf



Symptom	Engpass
Langsame Produktbilder	Storage I/O oder Netzwerk — große Binärdateien; fehlender CDN-Cache
Mobile API-Timeouts	Netzwerk — Latenz oder Bandbreitenproblem zwischen Client und Server

Übung 2.A.2: Vertikale vs. Horizontale Skalierung

Ein Start-up betreibt seinen Web-Shop auf einem einzelnen Server (8 Kerne, 32 GB RAM). Der Traffic wächst um Faktor 10.

1. Beschreiben Sie eine **vertikale** Skalierungsmaßnahme und nennen Sie ihre Grenze.
2. Beschreiben Sie eine **horizontale** Skalierungsmaßnahme und die dabei entstehenden neuen Herausforderungen.
3. Warum ist Session-State ein kritisches Problem bei horizontaler Skalierung, und wie löst man es?

Lösung

Modul B

Schichten und Architekturprinzipien

Übung 2.B.1: Schichtenzuordnung

Ein Entwickler implementiert eine Bestellfunktion. Ordnen Sie folgende Code-Fragmente den drei Schichten des 3-Schichten-Modells zu (Präsentation / Anwendung / Persistenz):

```
// (A)
<form action="/order" method="POST">
  <input name="productId" type="hidden" value="42"/>
  <button type="submit">Jetzt kaufen</button>
</form>

// (B)
if (product.getStock() < quantity)
  throw new InsufficientStockException();
double total = product.getPrice() * quantity * taxService.getRate();
emailService.sendConfirmation(user, order);

// (C)
@Query("SELECT p FROM Product p WHERE p.id = :id")
Optional<Product> findById(@Param("id") Long id);
```

Begründen Sie jede Zuordnung.

Lösung

Fragment	Schicht	Begründung
(A) HTML-Formular	Präsentation	Darstellung für den Nutzer, Eingabe-Erfassung — ändert sich wenn das Design ändert
(B) Lagerprüfung, Steuer, E-Mail	Anwendung	Geschäftsregeln, Validierung, Orchestrierung — ändert sich wenn Fachlogik sich ändert
(C) JPA-Query	Persistenz	Datenbankzugriff — ändert sich wenn die Datenbank ausgetauscht wird



Faustregel: Fragen Sie sich, welche Änderung diesen Code berühren würde. Designänderung → Präsentation. Neue Geschäftsregel → Anwendung. Anderes DB-Schema → Persistenz.

Übung 2.B.2: Architekturprinzipien anwenden

Beurteilen Sie folgenden Code anhand der Architekturprinzipien SoC, SRP und DRY:

```
@PostMapping("/checkout")
public String checkout(HttpServletRequest req, Model model) {
    String userId = req.getSession().getAttribute("userId").toString();
    ResultSet rs = db.query("SELECT * FROM cart WHERE user_id = " + userId);
    double total = 0;
    while (rs.next()) {
        total += rs.getDouble("price") * rs.getInt("qty") * 1.19;
    }
    // gleiche Steuerberechnung wie in /invoice und /quote
    String html = "<h1>Total: €" + total + "</h1>";
    model.addAttribute("content", html);
    sendEmail(userId, "Bestellung eingegangen, Total: €" + total);
    return "result";
}
```

1. Welche Prinzipien werden verletzt? Beschreiben Sie je eine konkrete Verletzung.
2. Skizzieren Sie eine verbesserte Struktur (Klassennamen genügen).

Lösung

Verletzungen:

- **SoC:** Der Controller enthält Datenbankzugriff (SQL), Geschäftslogik (Steuer), HTML-Erzeugung und E-Mail-Versand — vier verschiedene Verantwortlichkeiten in einer Methode.
- **SRP:** Die Methode tut mehr als eine Sache: sie verwaltet Session, berechnet Preise, generiert HTML und verschickt E-Mails.
- **DRY:** * 1.19 für die Steuer ist auch in /invoice und /quote dupliziert — bei Steueränderung müssen drei Stellen geändert werden.
- **SQL Injection:** "... WHERE user_id = " + userId ist eine Sicherheitslücke.

Verbesserte Struktur:

CheckoutController	→ nimmt Request entgegen, gibt View zurück
CartService	→ orchestriert Bestellprozess
TaxService	→ berechnet Steuer (eine Stelle, DRY)
CartRepository	→ Datenbankzugriff (PreparedStatement)
EmailService	→ E-Mail-Versand

Modul C

Verteilte Systeme

Übung 2.C.1: Herausforderungen verteilter Systeme

Ein Unternehmen migriert seinen Monolith auf 4 Server-Instanzen hinter einem Load Balancer.

Ordnen Sie folgende Vorfälle den Herausforderungen verteilter Systeme zu (Netzwerk-Latenz / Partielle Ausfälle / Konsistenz / Koordination):

Vorfall	Herausforderung
Server 2 fällt aus; 25 % der Requests schlagen fehl	?
Nutzer sieht nach dem Kauf kurz den alten Kontostand	?
Zwei Server verarbeiten gleichzeitig dieselbe Bestellung	?
API-Calls zwischen Services dauern 50 ms statt 0,1 ms	?

Lösung

Vorfall	Herausforderung
 Server 2 fällt aus	Partielle Ausfälle — einzelne Knoten können unabhängig versagen

Vorfall	Herausforderung
Alter Kontostand nach Kauf	Konsistenz — Replika noch nicht synchronisiert (eventual consistency)
Doppelte Bestellverarbeitung	Koordination — ohne Locking/Idempotenz bearbeiten mehrere Instanzen denselben Job
50 ms statt 0,1 ms	Netzwerk-Latenz — Netzwerkaufrufe sind Größenordnungen langsamer als lokale Operationen

Microservices und Web-Technologien

Übung 2.D.1: API Gateway und Fat Clients

Ein E-Commerce-Unternehmen hat seine Anwendung in 8 Microservices aufgeteilt. Die mobile App muss für eine einzige Produktseite Daten von 4 verschiedenen Services abrufen.

1. Was ist ein **API Gateway** und welches Problem löst es in diesem Szenario?
2. Worin unterscheidet sich ein **API Gateway** von einem einfachen **API Proxy (Reverse Proxy)**?
3. Diskutieren Sie **Fat Client vs. Thin Client**: Nennen Sie je zwei Vor- und Nachteile eines Fat Clients für Web-Anwendungen.

Lösung



1. **API Gateway:** Ein einziger Einstiegspunkt, der Client-Requests empfängt und intern an die richtigen Microservices weiterleitet. Es aggregiert Antworten, übernimmt cross-cutting concerns (Auth, Rate Limiting, Logging) und erspart dem Client das Wissen über die interne Service-Topologie. **Lösung:** Die App ruft nur das Gateway auf; dieses orchestriert die 4 Service-Calls intern.

2. Unterschied:

- **API Proxy / Reverse Proxy:** Leitet Anfragen 1:1 weiter ohne Transformation. Kein Routing-Intelligence, keine Aggregation.
- **API Gateway:** Bündelt, transformiert und aggregiert Anfragen. Kann Request-Composition (mehrere Services zu einer Antwort kombinieren), Auth-Enforcement, Rate Limiting und Response-Caching übernehmen.

3. Fat Client Vor- und Nachteile:

- **Vorteil:** Reaktionsschnelle UI (Logik im Browser), reduziert Server-Last
- **Vorteil:** Offline-Fähigkeit möglich (localStorage, Service Worker)
- **Nachteil:** Mehr JavaScript-Code → längere Ladezeit beim ersten Aufruf
- **Nachteil:** Sicherheits-kritische Logik darf nie nur im Client liegen (Client ist nicht vertrauenswürdig)

Übung 2.D.2: Web-Technologien kategorisieren

Ordnen Sie die folgenden Technologien/Standards in die richtige Kategorie ein:

Technologien: JSF, XML, HTTP, URI, HTML, CSS, SOAP, JSON, REST, Thymeleaf, JDBC, SQL

Kategorie	Technologien
Protokoll	?
Adressierung	?
Markup-/Datenformat	?
Framework / Template-Engine	?
Datenbankzugriff	?
Architekturstil	?

Hinweis: Manche Technologien können mehreren Kategorien zugeordnet werden — wählen Sie die primäre.

Kategorie	Technologien
Protokoll	HTTP, SOAP (baut auf HTTP auf), JDBC (Datenbankprotokoll)
Adressierung	URI
Markup-/Datenformat	HTML, XML, JSON, CSS
Framework / Template-Engine	JSF (Java Server Faces), Thymeleaf
Datenbankzugriff	JDBC, SQL
Architekturstil	REST

Anmerkungen:

- **SOAP** ist technisch ein Protokoll (XML-basiertes Messaging), baut aber typischerweise auf HTTP auf.
- **REST** ist kein Protokoll, sondern ein Architekturstil — ein häufiges Missverständnis.
- **XML** ist sowohl Datenformat als auch Basis für SOAP/WSDL-Beschreibungen.